

# **PDET Municipality Boundaries Dataset Integration**

## **Cuarta entrega**

Eliana Katherine Cepeda

Juan Diego Gonzalez

Mateo Moreno

Noviembre 17, 2025

## **1. Resumen Ejecutivo**

La Entrega 4 constituye la fase culminante del proyecto y tiene como objetivo central cumplir con el requerimiento principal de la UPME: cuantificar de manera precisa el potencial solar de los territorios PDET. Esto se logra mediante la estimación del número total de edificios y el área agregada de tejados para cada municipio PDET, comparando los resultados de dos fuentes de datos masivos: Microsoft Building Footprints y Google Open Buildings.

La metodología implementada en esta fase representa la evolución final del proceso, adoptando una estrategia de “Cargar–Luego–Analizar” (ELT). A diferencia de la aproximación inicial (Entrega 3), en la que se intentó filtrar por municipios antes de cargar y se encontraron serias limitaciones de memoria y tiempo de ejecución, en la versión final se optó por:

- Cargar en bruto todos los edificios de ambas fuentes a MongoDB (≈6 millones de Microsoft y ≈32 millones de Google), sin filtrado previo por municipio.
- Calcular el área únicamente para Microsoft en el momento de la carga (Google ya incluye el área en sus atributos) y unificar el formato de ambas fuentes para que tengan un esquema común en la base de datos (incluyendo el campo `area_m2`).
- Utilizar un script de Python que descarga la geometría de los 170 municipios PDET, construye de manera programática las consultas geoespaciales completas y las envía a MongoDB. El script ejecuta, en un bucle, 170 consultas (una por municipio) para cada fuente de datos.
- Debido a las limitaciones de la versión de MongoDB, el “join” entre los resultados de Microsoft y Google no se realiza dentro de Mongo (no se usa un \$lookup geoespacial complejo), sino en la capa de Python / Excel: los resultados agregados de cada fuente se combinan en una única tabla final a nivel de municipio.

La carga masiva de datos y el filtrado geoespacial se delegan al motor de MongoDB mediante índices 2dsphere y el operador \$geoIntersects, mientras que la integración final (join) entre fuentes y la generación del archivo Excel se realizan en Python.

De acuerdo con los requisitos del proyecto, los productos finales de este flujo de trabajo son:

1. Una estructura de datos tabulada (archivo .csv/.xlsx) que contiene, para cada municipio PDET, el número de edificios y el área total de techos estimada por fuente (Microsoft y Google).
2. Un conjunto de visualizaciones geoespaciales (mapas coropléticos) que traducen estos datos tabulados a un formato visual, permitiendo identificar patrones, densidades y contrastes del potencial solar sobre el mapa completo de Colombia.

## 2. Pasos Realizados

### 2.1. Correcciones respecto a la entrega anterior

| Antes                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Corrección                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• El ETL se planteó de forma ineficiente: se intentaba filtrar primero por municipios PDET en Python y luego cargar solo esos datos a MongoDB.</li><li>• El script de Python intentaba procesar millones de registros en memoria, lo que provocaba que el sistema se quedara sin RAM y “matara” el proceso (en Microsoft se detenía alrededor de las 458k filas y Google podía tardar más de 10 horas).</li><li>• Los datos cargados quedaban incompletos y, al usar un “molde unificado” parcialmente aplicado, no era posible identificar con claridad a qué municipio pertenecía cada edificio de manera sistemática.</li></ul> | <ul style="list-style-type: none"><li>• Se invirtió la lógica a un esquema ELT:<ul style="list-style-type: none"><li>◦ Se cargaron en bruto todos los edificios de Microsoft (~6 M) y Google (~32 M) a la base de datos, sin filtrar por municipio.</li><li>◦ Se homogenizó el formato de ambas fuentes, asegurando que todas las filas compartieran un mismo esquema (incluyendo <code>area_m2</code>).</li></ul></li><li>• Se implementó un script de carga robusto que inserta los datos en lotes (batches) y controla el uso de memoria.</li><li>• Se desarrolló un script de Python que:<ul style="list-style-type: none"><li>◦ Lee desde MongoDB los 170 municipios PDET (colección <code>municipios</code>) mediante un <code>.find({})</code>.</li><li>◦ Para cada municipio, extrae su geometría y construye el query geoespacial completo.</li><li>◦ Envía, en un bucle, 170 consultas por fuente a MongoDB usando <code>\$geoIntersects</code>, delegando el cálculo de conteos y sumas de áreas al servidor de base de datos.</li></ul></li><li>• Por limitaciones de la versión de MongoDB (que impide realizar joins geoespaciales avanzados directamente en el servidor), el join entre Microsoft y Google se realiza fuera de Mongo, en Python / Excel:<ul style="list-style-type: none"><li>◦ Python consolida los resultados por municipio en un <code>DataFrame</code>.</li><li>◦ Ese <code>DataFrame</code> se exporta a <code>.csv/.xlsx</code>, que es el archivo Excel “organizado” utilizado en los análisis finales.</li></ul></li></ul> |

## 2.2. Nuevo procesamiento de datos – Microsoft

Para Microsoft Building Footprints se siguió el siguiente flujo:

1. Insumo: archivo geoespacial (ej. GeoJSON) con los polígonos de edificios para Colombia.
2. Se cargó el insumo en GeoPandas y se verificó que la referencia espacial estuviera en coordenadas geográficas (EPSG:4326).
3. Para calcular el área de manera precisa, se reproyectó la geometría a un sistema de coordenadas planas apropiado para Colombia (ej. EPSG:3116).
4. Sobre la geometría reproyectada se calculó el área de cada edificio en metros cuadrados y se almacenó en un nuevo campo `area_m2`.
5. Se manejaron geometrías problemáticas o inválidas (por ejemplo, ignorando o corrigiendo casos puntuales) para garantizar que todas las áreas calculadas fueran consistentes.
6. Se construyeron los documentos a cargar en MongoDB con un esquema unificado, que incluye al menos:
  - Identificador de edificio (si aplica).
  - Campo fuente = "microsoft".
  - Campo `area_m2` (calculada).
  - Campo geometría en formato GeoJSON (en EPSG:4326), apto para índices 2dsphere.
7. Se cargaron todos los ~6 millones de edificios de Microsoft a MongoDB, en lotes, sin filtrar por municipio, únicamente normalizados y con el área calculada.

---

### *Nuevo Procesamiento datos Microsoft:*

```
Iniciando el proceso de ETL para el archivo: D:\Javeriana\Bases de datos\proyecto\microsoft\Colombia.geojson1
Progreso (Chunks): 0 chunk [00:00, ? chunk/s]
--- ¡PROCESO COMPLETO PARA MICROSOFT! ---
Tiempo total: 18.26 minutos
Total de filas procesadas: 6083821
Total de edificios cargados en MongoDB: 6083821
```

---

## 2.3. Nuevo procesamiento de datos – Google

Para Google Open Buildings se siguió un flujo similar, adaptado al formato disponible:

1. Insumo: archivo tabular (ej. CSV) con geometrías en formato WKT y atributos asociados a cada edificio.
2. Se cargó el CSV en Pandas y se convirtió la geometría WKT a objetos geométricos mediante Shapely.
3. Se integró esa geometría en un GeoDataFrame (GeoPandas) con CRS EPSG:4326.
4. A diferencia de Microsoft, Google ya proporciona una estimación del área del edificio (ej. campo `area_in_meters`). Por tanto:
  - No fue necesario recalcular el área;
  - Se normalizó ese campo para integrarlo como `area_m2` en el esquema final.
5. Se construyeron los documentos para MongoDB con el mismo esquema que Microsoft:
  - Campo fuente = "google".
  - Campo `area_m2` tomado del atributo de Google.
  - Campo geometría en formato GeoJSON (EPSG:4326).
6. Se cargaron todos los ~32 millones de edificios de Google a MongoDB, en lotes, sin filtrado previo por municipios, garantizando solamente la homogeneidad del formato.

### *Nuevo Procesamiento datos Google:*

```
--- Procesando Archivo 2/9: 8e1_buildings.csv ---
Progreso (Chunks): 0 chunk [00:00, ? chunk/s]
Archivo '8e1_buildings.csv' completado.

--- Procesando Archivo 3/9: 8e3_buildings.csv ---
Progreso (Chunks): 0 chunk [00:00, ? chunk/s]
Archivo '8e3_buildings.csv' completado.

--- Procesando Archivo 4/9: 8e5_buildings.csv ---
Progreso (Chunks): 0 chunk [00:00, ? chunk/s]
Archivo '8e5_buildings.csv' completado.

--- Procesando Archivo 5/9: 8e7_buildings.csv ---
Progreso (Chunks): 0 chunk [00:00, ? chunk/s]
Archivo '8e7_buildings.csv' completado.

--- Procesando Archivo 6/9: 8e9_buildings.csv ---
Progreso (Chunks): 0 chunk [00:00, ? chunk/s]
Archivo '8e9_buildings.csv' completado.

--- Procesando Archivo 7/9: 8ef_buildings.csv ---
Progreso (Chunks): 0 chunk [00:00, ? chunk/s]
Archivo '8ef_buildings.csv' completado.

--- Procesando Archivo 8/9: 91d_buildings.csv ---
Progreso (Chunks): 0 chunk [00:00, ? chunk/s]
Archivo '91d_buildings.csv' completado.

--- Procesando Archivo 9/9: 91f_buildings.csv ---
Progreso (Chunks): 0 chunk [00:00, ? chunk/s]
Archivo '91f_buildings.csv' completado.

--- ¡PROCESO COMPLETO! ---
Tiempo total: 80.89 minutos
Total de filas CSV procesadas: 31521921
Total de edificios cargados en MongoDB: 31521921
```

---

## **2.4. Consulta geoespacial por municipios PDET y join en Python**

Una vez todos los edificios estaban cargados en MongoDB:

1. El script de Python se conectó a la base de datos y consultó la colección municipios para obtener los 170 municipios PDET y sus geometrías.
2. El resultado de esa consulta se almacenó en una lista de Python, por ejemplo `municipios_pdet`, que funciona como la “lista de tareas”: el script sabe que debe repetir el análisis 170 veces, una por cada municipio.
3. Para cada municipio de esa lista, el script:
  - Extrae el polígono del municipio (ej. el polígono de “Tibú”).
  - Construye el filtro geoespacial usando `$geoIntersects` sobre el campo `geometria`.
  - Lanza dos pipelines de agregación hacia MongoDB (uno para Microsoft, otro para Google), con el siguiente esquema general:
  - Fase `$match` (Filtrado geoespacial)
    - Selecciona todos los edificios cuya geometría intersecta el polígono del municipio.
  - Fase `$group` (Agregación)
    - `"conteo": { "$sum": 1 } →` número total de edificios dentro del municipio.
    - `"area_total": { "$sum": "$area_m2" } →` suma del área de techos en metros cuadrados.

4. MongoDB devuelve así, para cada municipio y para cada fuente, un registro agregado con el conteo de edificios y el área total.
5. Por restricciones de la versión de MongoDB, estas agregaciones se ejecutaron por separado para cada fuente y no se utilizó un join geoespacial complejo en el servidor (como un \$lookup con pipeline geoespacial).
6. El join entre Microsoft y Google se realiza en Python:
  - El script va acumulando en memoria (por ejemplo, en un DataFrame de Pandas) las filas con: código DANE, nombre del municipio, fuente, conteo de edificios y área total.

Se manejan explícitamente los casos en los que Mongo no retorna resultados (por ejemplo, municipios sin edificios de una de las fuentes), rellenando esos valores con ceros para evitar errores (if stats\_google else 0).
7. Al finalizar el bucle sobre los 170 municipios, el DataFrame consolidado se exporta a archivo .csv/.xlsx.
  - Este archivo es el que se trabajó y organizó en Excel (el “join de los datos” que se construyó junto con Juan Diego), donde ya aparecen, para cada municipio PDET, las métricas finales de ambas fuentes.

```
¡Conexión exitosa a MongoDB! BD: 'is394508_db'
Cargando los 170 municipios PDET en la memoria...
Se encontraron 170 municipios.
Iniciando análisis geoespacial (esto puede tardar)...
Procesando Municipios: 0%|          | 0/170 [00:00<?, ?it/s]
--- ¡ANÁLISIS COMPLETO! ---
Tiempo total de procesamiento: 11.61 minutos
Mostrando los primeros 10 resultados:
municipio_nombre municipio_codigo_dane departamento google_conteo \
0      AMALFI          05031      ANTIOQUIA      2983
1      ANORÍ          05040      ANTIOQUIA      7652
2      APARTADÓ       05045      ANTIOQUIA     36337
3      BRICEÑO        05107      ANTIOQUIA     3565
4      CÁCERES        05120      ANTIOQUIA     12425
5      CAREPA         05147      ANTIOQUIA     16657
6      CAUCASIA       05154      ANTIOQUIA     30474
7      CHIGORODÓ      05172      ANTIOQUIA     18871
8      DABEIBA        05234      ANTIOQUIA     7696
9      EL BAGRE       05250      ANTIOQUIA     14768

google_area_total_m2 microsoft_conteo microsoft_area_total_m2
0      1.856396e+05      5184      7.596969e+05
1      4.945226e+05      5318      5.422179e+05
2      3.606578e+06      9477      2.011083e+06
3      2.090405e+05      2113      2.082718e+05
4      9.725477e+05      6762      6.848477e+05
5      1.726620e+06      7039      1.123826e+06
6      2.771754e+06     10208      1.737321e+06
7      1.883086e+06      7078      1.680870e+06
8      5.061714e+05      4766      4.949283e+05
9      1.314787e+06      7134      7.486756e+05

Resultados guardados en 'reporte_final_entrega_4.csv'
```

Además que maneja los casos donde no se encuentran edificios (ej. if stats\_google else 0) para evitar errores y rellenar con ceros. Termina de ejecutar el Script de Python y se guarda todo en un excel con los datos agrupados por municipio.

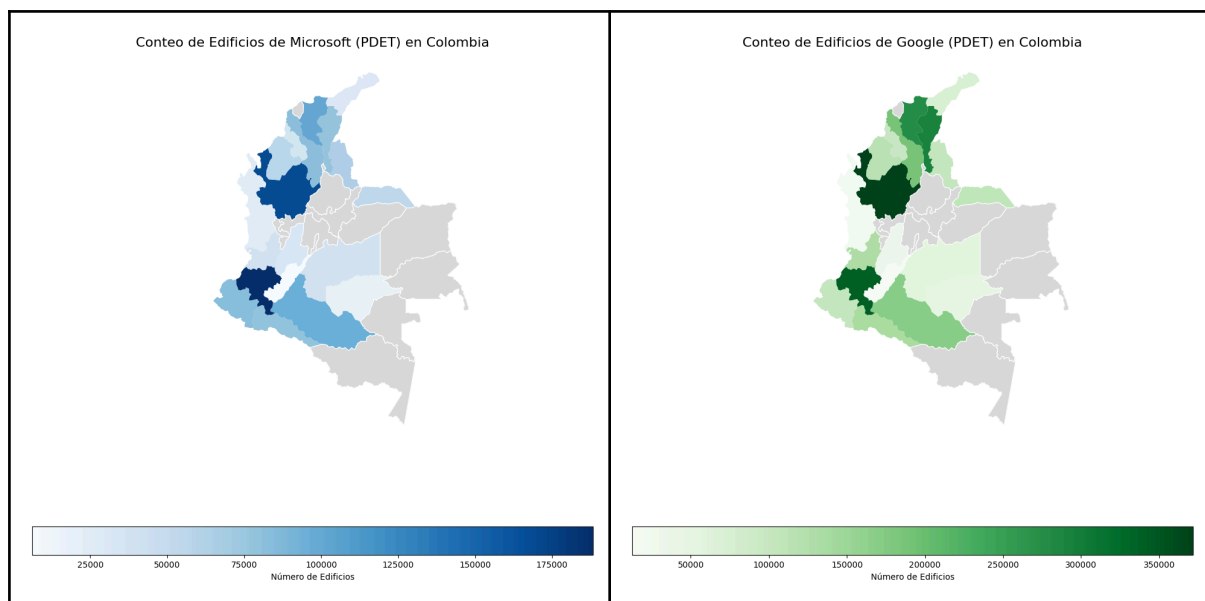
|    | A                  | B                     | C            | D             | E                    | F                | G                       |
|----|--------------------|-----------------------|--------------|---------------|----------------------|------------------|-------------------------|
| 1  | municipio_nombre   | municipio_codigo_dane | departamento | google_conteo | google_area_total_m2 | microsoft_conteo | microsoft_area_total_m2 |
| 2  | AMALFI             | 5031                  | ANTIOQUIA    | 2983          | 1856396093           | 5184             | 7,59697E+15             |
| 3  | ANORÍ              | 5040                  | ANTIOQUIA    | 7652          | 4945226483           | 5318             | 5,42218E+15             |
| 4  | APARTADÓ           | 5045                  | ANTIOQUIA    | 36337         | 3606578248           | 9477             | 2,01108E+15             |
| 5  | BRICEÑO            | 5107                  | ANTIOQUIA    | 3565          | 2090404883           | 2113             | 2,08272E+15             |
| 6  | CÁCERES            | 5120                  | ANTIOQUIA    | 12425         | 9725476676           | 6762             | 6,84848E+15             |
| 7  | CAREPA             | 5147                  | ANTIOQUIA    | 16657         | 17266204024          | 7039             | 1,12383E+16             |
| 8  | CAUCASIA           | 5154                  | ANTIOQUIA    | 30474         | 2771754468           | 10208            | 1,73732E+16             |
| 9  | CHIGORODÓ          | 5172                  | ANTIOQUIA    | 18871         | 18830855872          | 7078             | 1,68087E+16             |
| 10 | DABEIBA            | 5234                  | ANTIOQUIA    | 7696          | 5061714122           | 4766             | 4,94928E+15             |
| 11 | EL BAGRE           | 5250                  | ANTIOQUIA    | 14768         | 13147871012          | 7134             | 7,48676E+15             |
| 12 | ITUANGO            | 5361                  | ANTIOQUIA    | 9063          | 627500743            | 5776             | 6,00723E+15             |
| 13 | MURINDÓ            | 5475                  | ANTIOQUIA    | 0             | 0                    | 927              | 8,59907E+15             |
| 14 | MUTATÁ             | 5480                  | ANTIOQUIA    | 5537          | 4693866149           | 3118             | 3,41967E+15             |
| 15 | NECOCLÍ            | 5490                  | ANTIOQUIA    | 32173         | 23441543329          | 12879            | 1,21782E+16             |
| 16 | NECHÍ              | 5495                  | ANTIOQUIA    | 3492          | 2319781305           | 2852             | 2,5068E+16              |
| 17 | REMEDIOS           | 5604                  | ANTIOQUIA    | 13248         | 9630312446           | 8345             | 9,26883E+15             |
| 18 | SAN PEDRO DE URABÁ | 5665                  | ANTIOQUIA    | 22034         | 16447550511          | 9947             | 1,14195E+15             |
| 19 | SEGOVIA            | 5736                  | ANTIOQUIA    | 16158         | 119618533            | 7428             | 9,13841E+15             |

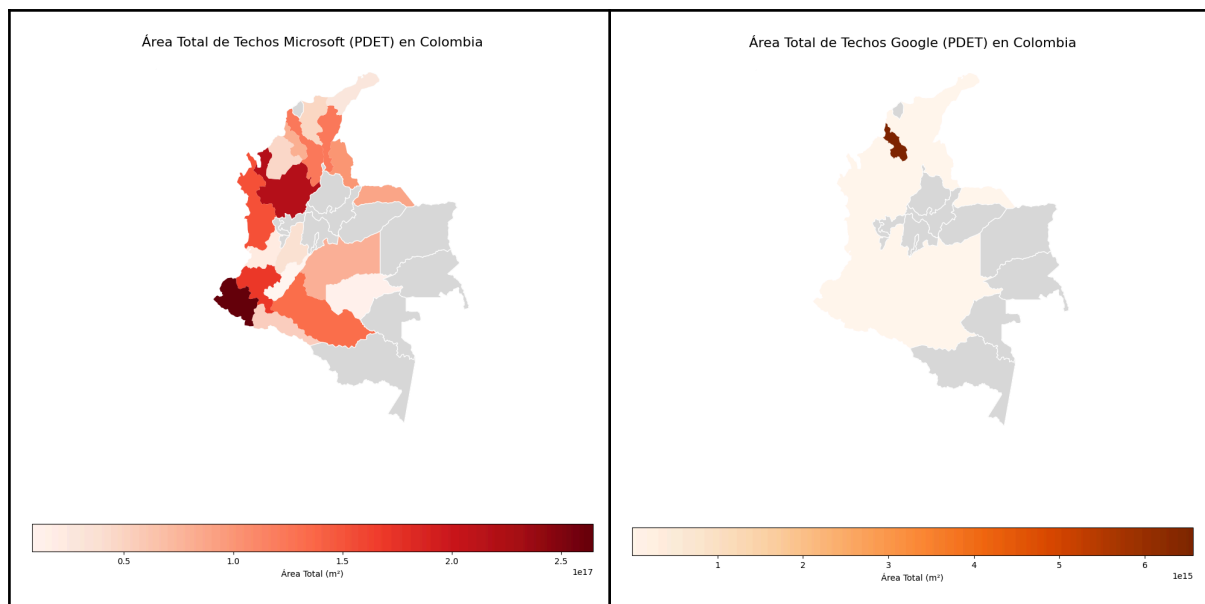
### 3. Visualizaciones Mapas:

A partir del archivo tabulado final (resultado de las consultas y del join en Python), se generaron las visualizaciones geoespaciales:

- Se unió la tabla de resultados con la capa geográfica de municipios PDET mediante el código DANE, de modo que cada municipio cuenta con sus atributos de:
  - número de edificios (Microsoft / Google),
  - área total de techos (Microsoft / Google),
  - y, cuando fue pertinente, indicadores derivados (ej. área promedio por edificio).
- Con esta capa enriquecida se construyeron mapas coropléticos, entre otros:
  - Mapas de conteo de edificios por municipio PDET (para cada fuente).
  - Mapas de área total de techos por municipio PDET.
  - Comparaciones visuales entre las dos fuentes para identificar diferencias sistemáticas.

Estos mapas permiten interpretar de forma intuitiva la distribución espacial del potencial solar y priorizar territorios para intervenciones posteriores.





## 4. Conclusiones

- La Entrega 4 corrigió las limitaciones de la aproximación inicial de la Entrega 3. El intento de “Filtrar–Luego–Cargar” (ETL) en Python resultó inviable por las restricciones de memoria y tiempo. La estrategia ELT (“Cargar–Luego–Analizar”) demostró ser mucho más estable y escalable.
- Cargar todos los edificios en bruto (Microsoft y Google) a MongoDB, calcular el área solo donde era necesario (Microsoft) y unificar el esquema, permitió trabajar con un repositorio centralizado y coherente de aproximadamente 38 millones de registros.
- El uso de consultas geospaciales (\$geoIntersects) y de índices 2dsphere en MongoDB trasladó el cálculo intensivo (filtrado y agregación) al motor de base de datos, reduciendo drásticamente los tiempos de ejecución respecto a un procesamiento puramente en Python.
- Debido a las limitaciones de la versión de MongoDB, el join entre las dos fuentes no se implementó dentro de la base de datos. En su lugar, el join se realizó en la capa de análisis (Python / Excel), combinando los resultados agregados de Microsoft y Google municipio por municipio. Esta solución híbrida fue práctica, reproducible y compatible con la infraestructura disponible.
- Se abordaron y manejaron adecuadamente desafíos técnicos como:
  - Cálculo de áreas en un sistema de coordenadas adecuado.
  - Manejo de geometrías problemáticas.
  - Control de casos sin edificios en una de las fuentes, evitando errores en el proceso de agregación y join.
- El flujo final de la Entrega 4 permitió generar:
  - Un reporte tabulado (.csv/.xlsx) con el número de edificios y el área total de los techos por municipio PDET, para ambas fuentes.
  - Las visualizaciones geospaciales necesarias para comunicar el potencial solar de los territorios PDET.